# Maximal Independent Set (MIS) Algorithm Performance Report

MIS Project Team

May 30, 2026

## 1 Introduction

This report evaluates various implementations of the Maximal Independent Set (MIS) algorithm. MIS is a fundamental problem in graph theory with applications in distributed computing, wireless networks, and resource allocation. We compare a sequential implementation against parallel versions based on Luby's algorithm, including multi-core CPU and GPU implementations.

## 2 Graph Representation

The efficiency of graph algorithms is heavily influenced by how the graph is stored in memory. In this project, we implemented and compared two primary representations:

### 2.1 Standard Adjacency List

The `Graph` type is implemented as a `std::vector<std::vector<uint32_t>>`. While this representation is intuitive and allows for easy graph construction, it has several drawbacks for high-performance computing:

- **Memory Fragmentation**: Each vertex's adjacency list is a separate memory allocation, leading to scattered data across the heap.
- **Cache Misses**: Traversing the neighbors of multiple vertices often results in frequent cache misses due to the lack of data contiguity.
- **Overhead**: Each `std::vector` object carries internal metadata (pointers, size, capacity), which adds significant memory overhead for large graphs with millions of vertices.

### 2.2 Compressed Sparse Row (CSR)

To address the limitations of the adjacency list, we implemented the `GraphCSR` class. This format stores the entire graph structure using two contiguous arrays:

- `offsets`: An array of size $N + 1$ where `offsets[i]` marks the starting position of vertex $i$'s neighbors in the `edges` array.
- `edges`: A single array of size $M$ (total number of edges) containing all neighbor indices stored back-to-back.

The CSR format offers several advantages:

- **Cache Locality**: Since all edges are stored contiguously, traversing neighbors is highly cache-efficient, significantly speeding up neighbor traversal phases.
- **Memory Efficiency**: CSR eliminates the metadata overhead of individual vectors and reduces memory fragmentation.
- **GPU Compatibility**: The contiguous nature of CSR makes it the standard format for graph processing on GPUs, as it allows for efficient coalesced memory accesses.

# 3 Algorithms

In this section, we describe the main algorithms implemented for the Maximal Independent Set (MIS) problem.

## 3.1 Sequential MIS

The sequential algorithm follows a greedy approach. It iterates through all vertices and adds a vertex to the MIS if none of its neighbors have been added yet. This algorithm has a linear time complexity of $O(N + M)$, where $N$ is the number of vertices and $M$ is the number of edges.

```
for (std::size_t i = 0; i < n; i++) {
    if (active[i]) {
        mis.push_back(i);
        for (auto &son : g[i])
            active[son] = 0;
    }
}
```

## 3.2 Luby's Algorithm

Luby's algorithm is a randomized parallel algorithm for finding a Maximal Independent Set. It is designed to work in a synchronous, round-based manner, making it highly suitable for parallel architectures like multi-core CPUs (via OpenMP) and GPUs (via CUDA).

The algorithm proceeds in discrete rounds, where each round consists of the following steps:

1. **Priority Assignment**: Each currently active vertex $v$ independently chooses a random priority $p(v)$ from a large uniform distribution.
2. **Local Maxima Selection**: A vertex $v$ is added to the independent set if its priority $p(v)$ is strictly greater than the priorities of all its active neighbors $N(v)$. Ties are typically broken using vertex indices.
3. **Pruning**: Any vertex selected in the previous step, along with all of its neighbors, is marked as inactive and removed from further consideration.

The process repeats on the remaining induced subgraph until no active vertices remain.

**Complexity**: Theoretically, Luby's algorithm is very efficient. It has been proven that the algorithm finds a Maximal Independent Set in $O(\log n)$ rounds with high probability, where $n$ is the number of vertices. Since each round involves traversing edges, the total work is proportional to the number of edges $M$, but the "depth" of the computation is logarithmic, allowing for significant speedups on parallel hardware.

### 3.2.1 OpenMP Parallelization

We utilized OpenMP to parallelize Luby's algorithm on multi-core CPUs, given these factors:

- **Shared Memory Model**: Luby's algorithm requires threads to frequently access the priorities and status of neighboring vertices. OpenMP's shared memory architecture allows all threads to access the global graph structure and priority arrays without the overhead of explicit message passing.
- **Data Parallelism**: The core operations of each round—priority assignment and local maxima selection—are naturally data-parallel. Using `#pragma omp parallel for` allowed us to distribute these tasks across available CPU cores with minimal code modification.
- **Dynamic Load Balancing**: In irregular graphs, such as Scale-Free networks, vertex degrees vary significantly. OpenMP's `dynamic` scheduling helps mitigate the "straggler" problem where threads processing high-degree nodes would otherwise bottleneck the entire round.
- **Synchronization Primitives**: OpenMP provides efficient primitives like `atomic` and `critical` sections, which were essential for managing shared state updates (e.g., adding nodes to the MIS or updating the "any active" flag) safely across threads.

### 3.3 Improved Luby's Algorithm

The improved version refines the base parallel Luby algorithm by reducing synchronisation overhead, eliminating explicit random number generation, and exploiting locality more effectively.

The key optimisations are:

- **Active Node Tracking**: Instead of scanning the entire vertex set each round, a dedicated vector `active_nodes` stores only those vertices still eligible for the independent set. After each round, survivors are compacted into a fresh list for the next iteration.
- **Deterministic, On-the-Fly Priorities**: Each vertex's random priority is computed from the current round number and its own index using a fast, splitmix64 hash function. This avoids an expensive thread-local RNG pass, eliminates the need for a global priority array, and guarantees a reproducible pseudo-random order without additional memory traffic.
- **Unified Max-Detection and Deactivation**: The search for local maxima and the immediate deactivation of winners and their neighbours are fused into a single parallel region. This fusion halves the number of global barriers per round compared to the basic version, reducing overhead.
- **Dynamic Load Balancing**: The neighbour-checking loop uses `schedule(dynamic, 256)` to adapt to the highly irregular degree distributions of scale-free and other non-uniform graphs.
- **Thread-Local Aggregation with Critical Merge**: Both the newly selected independent set vertices and the surviving active nodes are collected into thread-local lists during their respective loops. A single critical section per thread then appends the local results to a global vector. This pattern is used in the unified max-detection/deactivation region (for MIS additions) and again in the lightweight survivor compaction region.

### 3.4 Luby GPU Implementation

The GPU version of Luby's algorithm leverages the massive parallelism of CUDA architectures. The implementation is based on the CSR format and follows an iterative process where several kernels are launched per round:

- **Iterative Kernel Execution**: The algorithm operates in rounds. In each round, we launch a sequence of specialized CUDA kernels: one to identify candidates (local maxima) and another to update the MIS and prune neighbors. This cycle continues iteratively until all vertices have been processed.
- **Fused Kernels**: We combine priority assignment and candidate selection into a single kernel (`select_assign_priorities_fused_candidates_kernel`) to reduce global memory traffic and kernel launch overhead.
- **Restricted Pointers**: We utilize the `__restrict__` keyword for all input arrays (offsets, edges, active status). This informs the compiler that these pointers do not alias, enabling more aggressive load/store optimizations and better utilization of read-only data caches.
- **Deterministic Hashing**: Instead of generating and storing random numbers for all nodes in every round, we use a lightweight `hash_priority` function based on the vertex ID and iteration number. This reduces memory bandwidth requirements as priorities are computed on-the-fly.
- **Minimized Host-Device Transfers**: Only a single integer (`any_active`) is copied from device to host at the end of each round to determine convergence, keeping the hot loop mostly on the GPU.

## 4 Experimental Setup

To rigorously evaluate the performance of our implementations, we utilize two distinct random graph models that represent different topological characteristics commonly found in real-world applications.

### 4.1 Erdős-Rényi Model (Uniform Sparse)

The first model is the standard Erdős-Rényi random graph, specifically the $G(n, p)$ model [2]. In this model, each potential edge between any two vertices exists independently with a constant probability $p$. For our "Uniform Sparse" benchmarks, we set $p$ such that the average number of edges is $M = c \cdot N$, with $c$ typically set to 10.

This graph type serves as a baseline for "average" case performance. The degree distribution follows a Poisson distribution for large $n$, meaning most vertices have approximately the same number of neighbors. This relative uniformity simplifies workload balancing across parallel threads.

### 4.2 Barabási–Albert Model (Scale-Free)

The second model is the Barabási–Albert (BA) model [3], which generates scale-free graphs using a preferential attachment mechanism. The process starts with a small clique of $m_0$ nodes, and new nodes are added one by one, each connecting to $m$ existing nodes with a probability proportional to their current degree.

Scale-free graphs are characterized by a power-law degree distribution, $P(k) \sim k^{-\gamma}$. These graphs contain "hubs"—a few vertices with extremely high degrees—while the vast majority of vertices have very few connections. This topology is representative of social networks and the World Wide Web. For MIS algorithms, scale-free graphs present a significant challenge for parallelization due to extreme workload imbalance, making optimizations like dynamic scheduling and CSR layout essential.

## 5 Results

In this section, we present the performance results for the various implementations. All benchmarks were performed on a machine with an 8-core CPU (16 threads) and a modern GPU.

### 5.1 CSR vs Adjacency List Representation

We first justify the use of the Compressed Sparse Row (CSR) representation. Table 1 compares the execution time of sequential and Luby (8 threads) algorithms using both adjacency list and CSR formats on graphs with $10^6$ nodes.

Table 1: CSR vs Adjacency List Performance ($N = 10^6$)

| Algorithm | Graph Family | Adjacency List (ms) | CSR (ms) |
|---|---|---|---|
| Sequential | Uniform Sparse | 6.16 | 4.87 |
| Sequential | Scale-Free | 5.49 | 4.90 |
| Luby 8t | Uniform Sparse | 24.44 | 16.57 |
| Luby 8t | Scale-Free | 16.50 | 12.02 |

As expected, the CSR format consistently outperforms the standard adjacency list due to improved cache locality and reduced memory overhead. Figure 1 visualizes this difference.

### 5.2 Scaling with Graph Size

We evaluated the algorithms on graphs of varying sizes, ranging from $10^5$ to $2 \cdot 10^7$ nodes. The results are summarized in Table 2 and visualized in Figure 2.

### 5.3 Thread Scaling

We investigated the parallel efficiency of Luby and Improved Luby algorithms by varying the number of threads. Figure 3 shows the results for $N = 5 \cdot 10^6$. Improved Luby shows significantly better scaling and lower overhead compared to the base implementation.

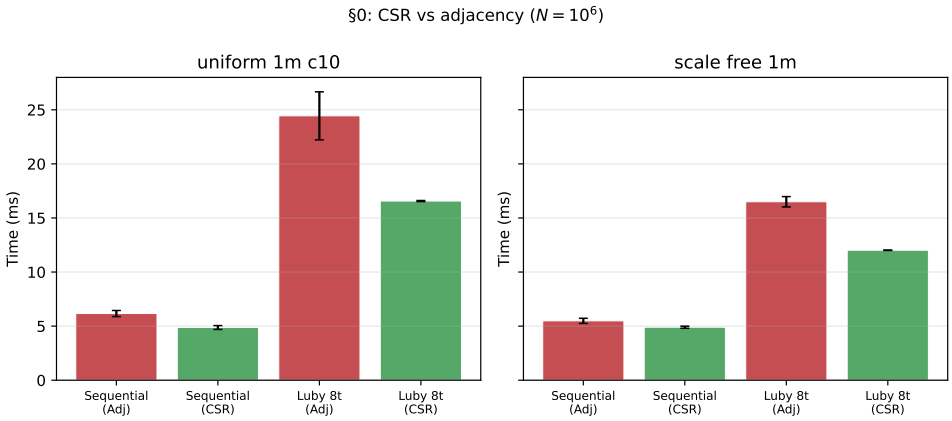


Figure 1: Performance comparison between CSR and standard adjacency list representation.

Table 2: Performance scaling with problem size $N$ (CSR representation)

| $N$ | Sequential (ms) | Luby 8t (ms) | Luby Improved 8t (ms) | Luby GPU (ms) |
|---|---|---|---|---|
| | | *Uniform Sparse* ($c = 10$) | | |
| $10^5$ | 0.49 | 1.11 | 0.84 | 5.40 |
| $10^6$ | 4.65 | 15.79 | 8.68 | 11.88 |
| $5 \cdot 10^6$ | 34.65 | 129.25 | 50.07 | 65.35 |
| $10^7$ | 74.86 | 305.24 | 103.28 | 142.11 |
| | | *Scale-Free* ($m = 5$) | | |
| $10^5$ | 0.44 | 0.98 | 0.74 | 0.99 |
| $10^6$ | 4.64 | 11.44 | 6.70 | 7.82 |
| $5 \cdot 10^6$ | 27.61 | 87.22 | 37.65 | 39.91 |
| $10^7$ | 60.45 | 205.54 | 78.02 | 86.20 |
| $2 \cdot 10^7$ | 247.66 | 515.66 | 229.32 | 301.81 |

### 5.4 Impact of Graph Topology

The degree distribution significantly impacts parallel performance. Table 3 shows results for different densities ($c$) and scale-free parameters. The GPU implementation shows its strength on denser graphs, while Improved Luby handles highly irregular scale-free graphs effectively.

Table 3: Performance comparison across different graph topologies ($N = 5 \cdot 10^6$)

| Topology | Sequential (ms) | Luby 8t (ms) | Improved 8t (ms) | GPU (ms) |
|---|---|---|---|---|
| Uniform $c = 5$ | 29.23 | 97.35 | 36.38 | 43.91 |
| Uniform $c = 50$ | 47.58 | 183.48 | 109.22 | 307.42 |
| SF $m = 3$ | 26.47 | 68.88 | 29.39 | 32.15 |
| SF $m = 10$ | 36.36 | 120.52 | 47.71 | 64.66 |

## 6 Weighted MIS

### 6.1 Problem Formulation

We recall that the standard Maximal Independent Set (MIS) is a subset of vertices $S \subset V$ of a graph $G = (V, E)$ such that no two vertices in $S$ are joined by an edge, and no new vertex can be added to $S$ without creating an edge. Now, we extend this framework by adding a weight $w_v \in \mathbb{R}^+$ for every vertex $v \in V$. In this setting, the goal is now to find a MIS with the maximum possible total weight **in practice**. It is important to note that finding the **maximum weight independent set** (i.e., the

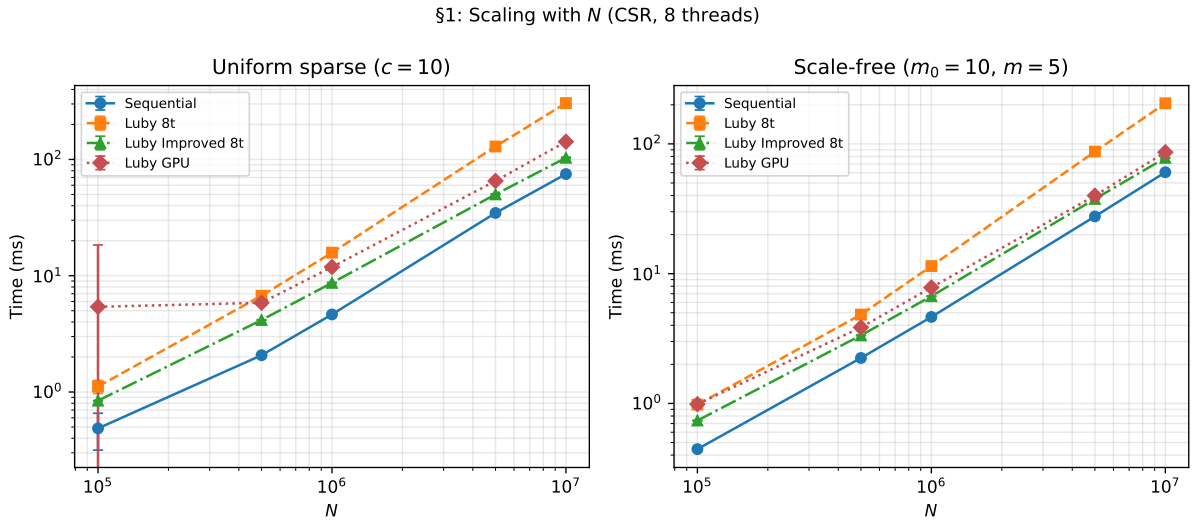


Figure 2: Scaling of execution time with the number of vertices $N$ for different algorithms.

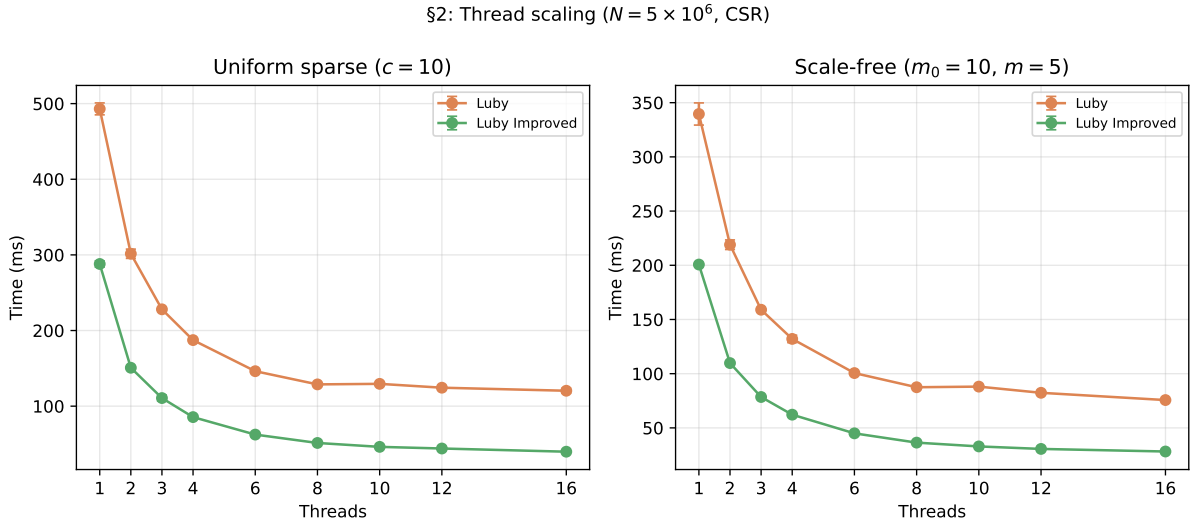


Figure 3: Performance scaling with the number of OpenMP threads ($N = 5 \cdot 10^6$).

globally optimal solution) is NP-hard in general. In this view, we will focus on weight-based heuristic algorithms that find a valid MIS while favoring vertices with high weights, and then compare their solution quality empirically.

## 6.2 Algorithms

We decided to study the performance of the following two algorithms, and then benchmark their performance based on the initial weight assignment.

### 6.2.1 Algorithm 1: Weighted Greedy

A vertex $v$ is added to the MIS if and only if it has the highest weight among its active neighbors:

$$v \text{ wins} \iff w_v \geq w_u \quad \forall u \in N(v),\ u \text{ active} \tag{1}$$

Ties are broken by vertex index. This approach is based on Basagni [1].

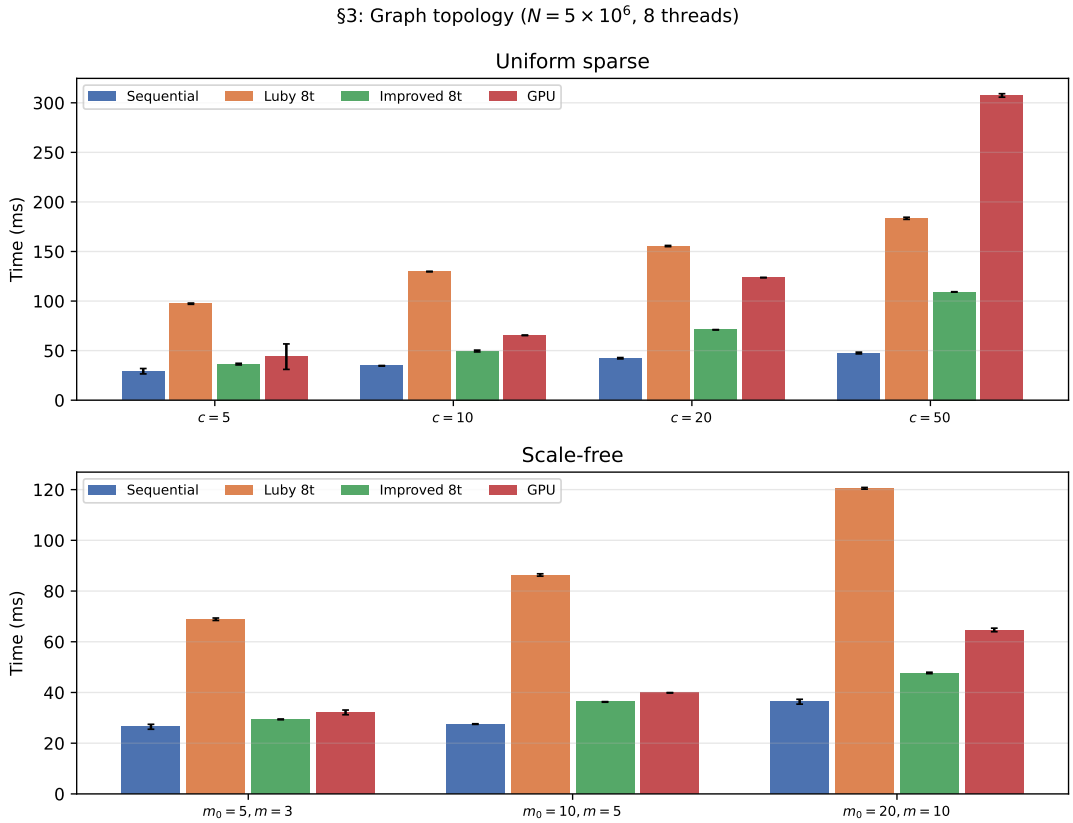


Figure 4: Effect of graph density and scale-free parameters on algorithm performance.

#### 6.2.2 Algorithm 2: Weighted Sampling

Each active vertex $v$ draws $U_v \sim \text{Uniform}(0, 1)$ and computes a priority:

$$p_v = U_v^{1/w_v} \tag{2}$$

Vertex $v$ wins the round if $p_v > p_u$ for all active neighbors $u$. That is, we have that the vertices with higher weights have the higher priority.

### 6.3 Weight Distributions

**Uniform:** Each vertex $v$ is assigned a weight drawn uniformly at random:

$$w_v \sim \text{Uniform}(1, 10) \tag{3}$$

**Exponential:** Each vertex $v$ is assigned:

$$w_v = 1 + X, \quad X \sim \text{Exponential}(\lambda) \tag{4}$$

The $+1$ shift ensures $w_v > 0$, which is required by the power trick in Algorithm 2.

**Clustered:** A fraction $\rho = 0.1$ of vertices are randomly designated as hotspots:

$$w_v = \begin{cases} 10 & \text{with probability } \rho \\ 1 & \text{otherwise} \end{cases} \tag{5}$$

### 6.4 Experimental Results

The following results were captured from the weighted MIS benchmark suite ($N = 1{,}000{,}000$, 5 graphs, 10 runs each).

| Algorithm | Graph Type | Mean Time | Graph Std | Avg Run Std | Mean Weight |
|---|---|---|---|---|---|
| Weighted Greedy | Uniform Sparse (uniform) | 48.346 ms | 10.319 ms | 19.023 ms | 221,205.58 |
| Weighted Sampling | Uniform Sparse (uniform) | 101.725 ms | 62.736 ms | 86.907 ms | 169,862.79 |
| Weighted Greedy | Scale-Free (uniform) | 34.835 ms | 5.984 ms | 7.885 ms | 2,220,850.60 |
| Weighted Sampling | Scale-Free (uniform) | 65.297 ms | 47.481 ms | 41.514 ms | 1,998,436.76 |
| Weighted Greedy | Uniform Sparse (exp) | 44.856 ms | 2.182 ms | 7.825 ms | 183,315.81 |
| Weighted Sampling | Uniform Sparse (exp) | 65.743 ms | 33.578 ms | 24.710 ms | 102,926.19 |
| Weighted Greedy | Scale-Free (exp) | 53.372 ms | 44.933 ms | 35.131 ms | 1,338,124.13 |
| Weighted Sampling | Scale-Free (exp) | 40.659 ms | 5.625 ms | 11.284 ms | 1,133,123.09 |
| Weighted Greedy | Uniform Sparse (clustered) | 63.872 ms | 4.375 ms | 7.339 ms | 163,565.00 |
| Weighted Sampling | Uniform Sparse (clustered) | 45.382 ms | 3.298 ms | 5.214 ms | 107,985.08 |
| Weighted Greedy | Scale-Free (clustered) | 41.219 ms | 12.018 ms | 25.399 ms | 1,051,920.60 |
| Weighted Sampling | Scale-Free (clustered) | 62.458 ms | 25.564 ms | 44.257 ms | 877,655.86 |

### 6.5 Discussion

The results indicate that the Greedy approach consistently finds the MIS with higher total weight across all graph types and initial weight distributions. This can be explained by the fact that stochastic sampling can occasionally make the algorithm give a low-weight neighboring vertex a higher priority, thus permanently excluding the high-weight vertex from the MIS selection.

We also see that on Scale-Free graph types, the mean weight results are much closer than those on the Uniform-Sparse graph types (gap of 11–20% on Scale-Free vs. 30–78% on Uniform Sparse).

We see that the exponential and clustered initial weight distributions improve greedy's advantage (78% and 52% gaps respectively on Uniform Sparse, vs. 30% for uniform weights). The reason for this is that in these settings, we have relatively fewer very high-weight vertices, and the greedy approach always captures them. Meanwhile, the sampling approach may exclude them in case a low-weight vertex is selected instead at random.

## References


[1] S. Basagni. Finding a maximal weighted independent set in wireless networks. Telecommunication Systems, 18(1–3):155–168, 2001. URL http://dx.doi.org/10.1023/A:1016747704458.

[2] Erdős, P. and Rényi, A., 1959. On random graphs I. Publ. Math. Debrecen, 6, pp.290-297.

[3] Barabási, A.L. and Albert, R., 1999. Emergence of scaling in random networks. Science, 286(5439), pp.509-512.